



INSTITUTE OF COMPUTER SCIENCE & DIGITAL INNOVATION

Assignment 1

**Agile software development methods (Scrum, Extreme Programming, Lean
Software Development)**

Course : Object Oriented Methods – BIC3133

Semester : February 2026

Lecturer : Dr. Kaviya Elakkiya Murugesan

Date of submission: 20 February 2026

Student Name	Student Id
Hasna E Noor Rahman Irfan	1002370486

Table of Contents

1.0 Introduction	3
1.1 Scrum	3
1.2 Extreme Programming.....	3
1.3 Lean Software Development.....	4
2.0 Characteristics	4
2.1 Scrum	4
2.2 Extreme Programming.....	4
2.3 Lean Software Development.....	5
3.0 Principles	5
3.1 Scrum Principles.....	5
3.2 Extreme Programming Principles	5
3.3 Lean Software Development Principles	6
4.0 Process	6
4.1 Scrum Process	6
4.2 Extreme Programming Process	7
4.3 Lean Software Development Process	7
5.0 Strengths and Weaknesses	8
5.1 Scrum Strengths and Weaknesses	8
5.2 Extreme Programming Strengths and Weaknesses	8
5.3 Lean Software Development Strength and Weaknesses	9
6.0 Tools and Techniques.....	10
6.1 Scrum Tools and Techniques	10
6.2 Extreme Programming Tools and Techniques.....	10
6.3 Lean Software Development Tools and Techniques.....	11
7.0 Diagrams	12
7.1 Scrum Diagram	12
7.2 Extreme Programming Diagram	13
7.3 Lean Software Development Diagram	14
8.0 Case Study.....	14
References	16
Appendix	17

1.0 Introduction

Agile software development practices have been created in reaction to the already reported limitations of more traditional plan-driven approaches such as the Waterfall model. The old sequential-based approaches were strict, phase-based and therefore the requirements were first collected, then designed, implemented, tested and finally delivered; this meant that accommodating customer changes was very hard, expensive and time consuming. These models tended to rely on detailed documentation. This inflexibility resulted in inadequate feedback and massive rework expenses and inappropriateness to stakeholder requirements. Agile approaches, on the other hand, focus on the iterative delivery process, constant customer engagement, and flexibility to change, which enables the teams to release functional software faster and adapt to changes as requirements change during the development. (Contributor, 2021)

1.1 Scrum

It starts with understanding the Scrum framework which was first introduced to the world in 1995 as a better way of team collaboration for solving complex problems. The Scrum framework is simple being made up of a Scrum Team consisting of a Product Owner, a Scrum Master and Developers, each of which have specific accountabilities. The Scrum Team takes part in five events and produces three artifacts. So why is it called Scrum? People often ask, “Is Scrum an acronym for something?” and the answer is no. It is inspired by a scrum in the sport of rugby. In rugby, the team comes together in what they call a scrum to work together to move the ball forward. In this context, Scrum is where the team comes together to move the product forward. (Scrum.org., 2026)

1.2 Extreme Programming

Extreme Programming (XP) focuses on engineering discipline and technical excellence to improve software quality. Unlike other agile methods that focus primarily on project organization and process, XP introduces a set of core engineering practices such as test-driven development (TDD), pair programming, continuous integration, and frequent releases to ensure frequent feedback and quality. Recent comparative studies show that XP is often chosen in projects where responsiveness to changing requirements and high technical standards are priorities. (Asma Akhtar, 2022) (Abdulrahman Alsari, 2023)

1.3 Lean Software Development

Lean Software Development originally developed to eliminate waste and optimize value delivery to the context of software engineering. Although lean thinking began within manufacturing decades earlier, it was formalized for software development in the early 2000s and later integrated with agile practices. Lean emphasizes maximizing value by eliminating non-essential activities, improving flow, and empowering teams. Research on agile and lean integration highlights how Lean principles continue to shape agile practices by driving efficiency and value focus across development processes. Today, Lean is recognized as both a complement and a foundation for many agile approaches that aim to deliver value with minimal waste. (Felix Temole, 2025)

2.0 Characteristics

2.1 Scrum

Key characteristics:

1. Time-boxed sprint cycles with frequent feedback
2. Defined roles supporting collaboration and accountability,
 - a. Project Owner
 - b. Scrum Master
 - c. Development Team
3. Prescribed events for planning, inspection, and adaptation
4. Incremental delivery of potentially shippable software
5. Empirical process control through transparency and adaptation (Michal Hron, 2022)

2.2 Extreme Programming

Key characteristics:

1. Short iteration cycles with frequent deliverables
2. Quality and reliability Test-driven development (TDD).
3. Code quality and collaborative programming Co-opetition Pair programming.
4. Frequent integration of work through continuous integration.
5. Active customer involvement in order to optimize requirements. (Abdulrahman Alsari, 2023)

2.3 Lean Software Development

Key characteristics:

1. Concentrate on eradication of wastage in the development process.
2. Constructing quality into every-day operation, not looking at the end.
3. Delays commitment to minimize risk and enhancement of flexibility.
4. Move fast to become more responsive to the change.
5. Enable teams and streamline the entire process.

3.0 Principles

3.1 Scrum Principles

Based on current agile studies, Scrum has its backbone in empirical process control with the focus on the major principles of transparency, inspection and adaptation. The following principles can assist teams in dealing with complexity through the regular review of work output, changing plans according to feedback, and keeping track of progress through the articulation of artifacts and events. The latter principles allow Scrum teams to continuously enhance their performance and adapt to new requirements. (Adrielle Cristina Sassa, 2023)

Key Scrum Principles:

Transparency: All work and decisions are visible to stakeholders.

Inspection: Work is frequently assessed for deviations.

Adaptation: Changes are made promptly based on feedback.

3.2 Extreme Programming Principles

The main principles of XP are constant customer engagement and feedback loops, which facilitate change responsiveness and quality software results. Such methods as Test-Driven Development (TDD) and pair programming are the manifestations of these concepts, reinforcing the quality and collaboration. (Felix Temole, 2025)

Key XP Principles:

Rapid feedback: Constant customer and team feedback loops.

Technical excellence: Practices like TDD and pair programming to enhance code quality.

Continuous involvement: The customer participates actively throughout development.

3.3 Lean Software Development Principles

The lean concept of software is geared towards ensuring the best value delivery with minimum waste. Recent studies on agile integration point out to the fact that the principles used by the Lean system like removal of waste, rapid delivery, team empowerment and optimization of the entire system facilitate the development process and leverage of responsiveness to customer requirements. These concepts are based on lean manufacturing and other agile conditions in software development to minimize delays and add value. (Felix Temole, 2025) (Lynn, 2025)

Key Lean Principles:

Eliminate waste: Remove non-value-adding activities.

Deliver fast: Shorter cycles to improve adaptability.

Empower teams: Decentralize decision-making to enhance performance.

Optimize the whole: Focus on system-wide efficiency rather than individual tasks.

4.0 Process

4.1 Scrum Process

Scrum process has a planned yet adaptable cycle that aims at providing incremental value through continuous repetitions. The normal Scrum process starts with Product Backlog where all features and requirements are listed and ranked. Sprint Planning is done by the team before every sprint to determine the backlog items that it will work on. A Sprint (typically 1-4 weeks) is then time-boxed to work on those items by the team, and there is a Daily Scrum meeting daily to manage the work and solve impediments. The Sprint Review at the end of the sprint is a presentation of what is done to the stakeholders, and the Sprint Retrospective is the reflection of the process and how future sprints should be improved. This is repeated until the project goals are achieved with the backlog being continually refined to respond to change. The studies

of Agile practices prove the fact that the structured process of Scrum is based on iterative planning, daily synchronization, and continuous inspection and adaptation. (Maria Spichkova, 2025)

4.2 Extreme Programming Process

The XP process emphasizes frequent iterations and strong engineering discipline to ensure high software quality and responsiveness to change. In XP, the project is broken into short development cycles (often weekly or bi-weekly), during which teams work on small, well-defined tasks. Each cycle includes planning at the start, collaborative development using practices such as pair programming, test-driven development (TDD), and continuous integration, and frequent feedback from the customer. XP encourages releasing working software frequently and continuously revisiting priorities, with customer involvement throughout to refine requirements and verify that the evolving system meets needs. Research reviews indicate that XP's iterative feedback loops and disciplined engineering practices form the backbone of its process. (Rania Aleryani, 2025)

4.3 Lean Software Development Process

Lean Software Development does not prescribe a fixed series of stages like Scrum or XP instead; it focuses on optimizing the flow of value through the software delivery process. The process begins with value stream mapping to identify every activity involved from concept to delivery and pinpoint non-value-adding steps (waste). Teams then apply Lean principles such as eliminating waste, amplifying learning, delivering fast, and optimizing the whole to streamline work. Continuous improvement (Kaizen) is embedded, with teams regularly reflecting and refining practices to reduce delays and improve flow. While Lean itself is more of an overarching approach than a prescriptive SDLC, recent research on Lean adoption in software emphasizes visualizing work, reducing cycle times, and improving value delivery through reduced waste and efficient workflows. (Buehring, 2024)

5.0 Strengths and Weaknesses

5.1 Scrum Strengths and Weaknesses

Strengths

High Flexibility: Scrum allows teams to adapt quickly to changing requirements through short sprint cycles and regular backlog refinement.

Improved Transparency: Defined roles, events, and artifacts (like Sprint Review, Daily Scrum) ensure visibility of progress and accountability.

Frequent Feedback: Sprint Reviews provide regular stakeholder involvement, reducing misunderstandings and increasing customer satisfaction.

Continuous Improvement: Sprint Retrospectives encourage teams to reflect and improve performance continuously.

Incremental Value Delivery: Each sprint produces a potentially usable product increment, reducing risk.

Weaknesses

Requires Experienced Team Members: Self-organizing teams need discipline and strong collaboration skills.

Scope Creep Risk: Frequent backlog changes can cause instability if not properly managed.

Not Ideal for Large Distributed Teams: Daily coordination may become complex in geographically dispersed teams.

Role Confusion: Misunderstanding between Scrum Master and Project Manager roles can cause conflict.

5.2 Extreme Programming Strengths and Weaknesses

Strengths

High Software Quality: Practices such as Test-Driven Development (TDD) and continuous integration reduce defects.

High Customer Involvement: The system is driven with continuous customer feedback to ensure that it addresses real needs.

Quick Response to Change: Requirement updates can be quickly done due to short iterations.

Better Team Cooperation: Pair programming increases the sharing of knowledge and mistakes are minimized.

Less Technical Debt: Refactoring ensures a lean code.

Weaknesses

Discipline is High: such software development practices as TDD and pair programming require regular discipline.

Customer Availability Requirement: We shall not always be able to have continuous customer presence.

More Initial Cost: Pair programming may seem to be expensive with respect to manpower.

Unsuitable where Large Projects should be Scaled: XP is most effective in small to medium-sized teams.

5.3 Lean Software Development Strength and Weaknesses

Strengths

Eliminates Waste: Focuses on removing non-value-adding activities, improving efficiency.

Better Delivery: Efficiencies and quicker workflow make work shorter and more responsive.

Efforts to Optimize Entire System: Stimulates enhancement of the entire process, not only of the individual tasks.

Fosters lifelong Learning: Stimulates innovation and team empowerment.

Efficiency in costs: Wastes reduce the development costs.

Weaknesses

Less Prescriptive Organization: Lean offers rules as opposed to procedures, which can be confusing.

Needs Cultural Change: Organizations have to embrace a culture of efficiency and empowerment.

Complexity of Measurement: It may be difficult to identify and measure the amount of waste.

Threat of Over-Optimization: Too much emphasis on efficiency can lower the level of creativity.

6.0 Tools and Techniques

6.1 Scrum Tools and Techniques

Tools

Jira: Includes Scrum templates for backlog refinement, sprint planning, task prioritization, and reporting. (Atlassian, 2025)

Trello: A Kanban-style board useful for visual task tracking and workflow which supports Scrum team planning. (Tersoo Hulugh, 2024)

Techniques

User Story Backlog Management: Organizing features into prioritised user stories for sprint planning.

Sprint Planning & Task Estimation: Using tools like Jira to plan sprints and break down work. (Oleksandra Svintsytska, 2025)

Sprint Boards & Burndown Charts: Key visualization techniques to monitor progress in a sprint. (Atlassian, 2025)

6.2 Extreme Programming Tools and Techniques

Tools

Test Automation Tools: Support Test-Driven Development (TDD) and regression testing. Best options include frameworks like JUnit, Selenium, or CI tools.

Techniques

Test-Driven Development (TDD): Write tests before code to ensure reliability.

Pair Programming: Two developers work together to improve quality and knowledge sharing.

Continuous Integration (CI): Frequently merge code into shared repository to detect integration issues early. (Pham, 2025)

Frequent Releases: Small frequent delivery cycles to continuously validate functionality.

6.3 Lean Software Development Tools and Techniques

Tools

Kanban Boards: Visualize workflow and limit work in progress, derived from lean principles. (Rehkopf, 2025)

Value Stream Mapping: Visual tool to identify value-adding vs non-value-adding activities.

Techniques

Waste Elimination: Stop activities that don't deliver customer value.

Flow Optimization: Tracking cycle time to improve delivery speed and reduce bottlenecks.

Empowering Teams: Teams manage their work and decisions to build value faster.

Continuous Improvement: Iterative adjustments to workflows and practices. (Rehkopf, 2025)

7.0 Diagrams

7.1 Scrum Diagram

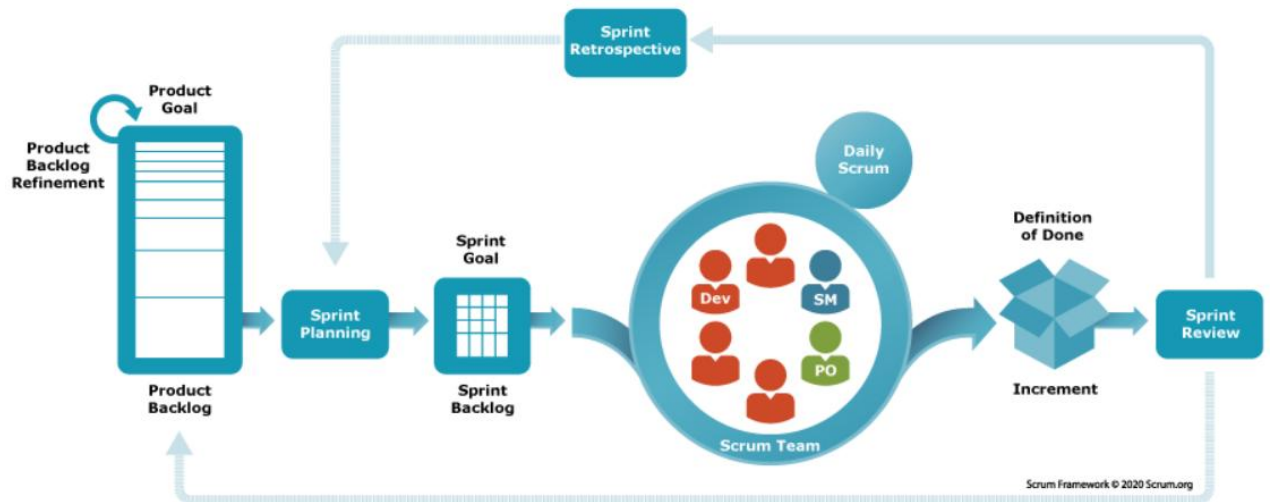


Figure 1: Scrum workflow diagram (Scrum.org., 2026)

It starts with the Product Backlog that is composed of all the features, requirements, and improvements ordered in terms of their priorities by the Product Owner. Then follows Sprint Planning where the team picks items in the backlog and develops a Sprint Goal. The team engages in the selected tasks in order to develop a possibly usable product increment. To coordinate work a Daily Scrum meeting is conducted every day. The team also presents the working increment to the stakeholders and gets feedback at the end of the sprint. The team reviews what has been done well and what can be done better. Subsequently a prototype version of the product is provided. And the process repeats to the backlog to another sprint.

7.2 Extreme Programming Diagram

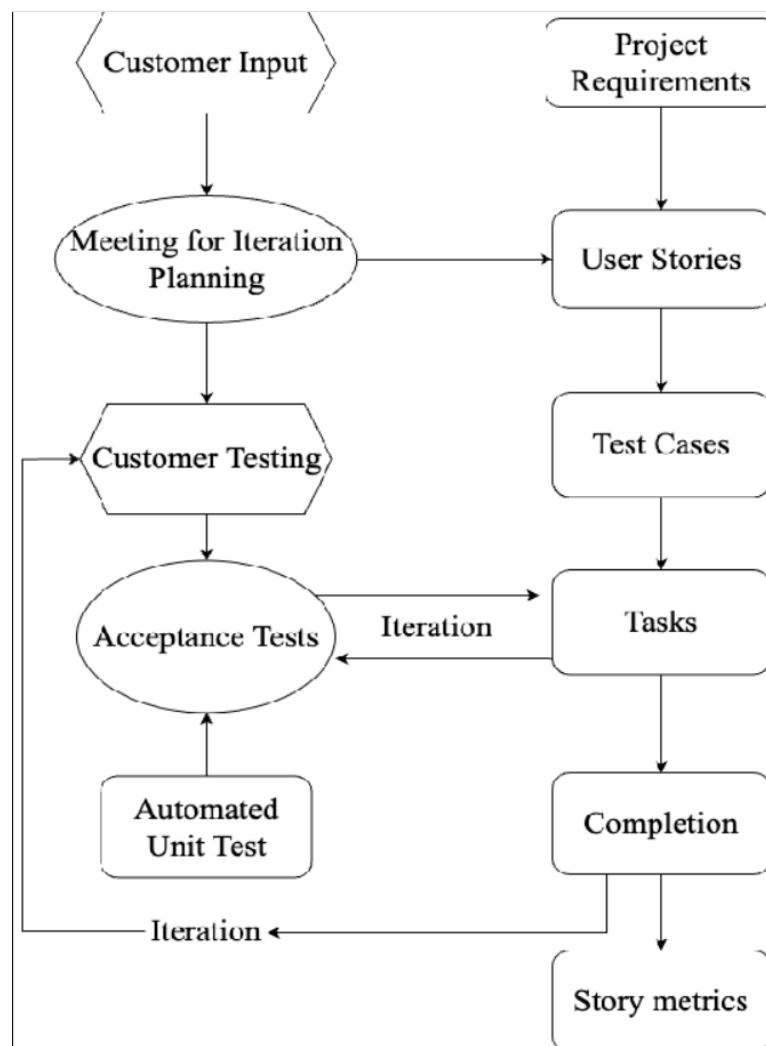


Figure 2: XP workflow

This XP workflow diagram depicts an iterative development cycle that is customer-driven and is fuelled by testing. It starts with customer input and project requirements which are translated into user stories in iteration planning. These are separated into tasks and test cases and the developers perform them using automated unit testing. The system is then accepted and tested by the customer and feedback could lead to yet another cycle of enhancement. After meeting the requirements, the work is accomplished and it is measured through story metrics. The diagram emphasizes on the emphasis of XP on short iterations, on-going testing, and frequent customer feedback.

7.3 Lean Software Development Diagram

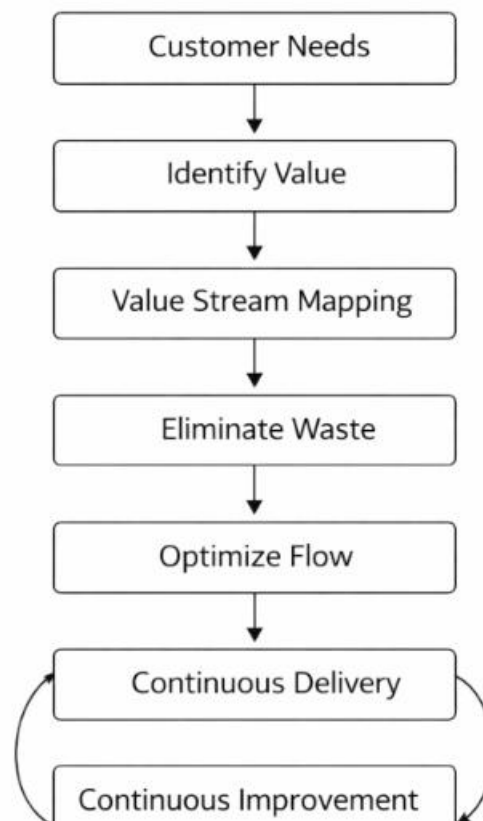


Figure 3: Lean Software Development workflow

This Lean Software Development process is a representation of the way value is provided and refined. This starts with getting to know the needs of the customers and what actually brings value. The team then maps the value stream to have a visual representation of all the steps followed and get rid of any waste (non-value-adding activity). Once the inefficiencies are eliminated, the attention is paid to streamlining of the workflow to achieve a smooth and expeditious delivery. The product delivery is continuous and the cycle is repeated by continuous improvement (Kaizen) in order to increase efficiency and quality as time goes by.

8.0 Case Study

Scrum Implementation in Development of Smart Start E-Commerce Website

A recent empirical research examined the use of Scrum framework in a practical project involving the development of an e-commerce. The study was about the application of Scrum to facilitate project efficiency, quality and customer satisfaction in the development of the

Smart Start e-commerce site. Scrum was used in all the software lifecycle stages which started with the formation of a Product Backlog and followed by Sprint Planning, Sprint execution, Daily Standups, Sprint Review, and Sprint Retrospective. This organized Scrum process has allowed the project team to divide work into small manageable increments and involve stakeholders in the process of development constantly.

During the project lifecycle, a total of 54 story points were completed across multiple sprints, and Blackbox testing was used to verify that all delivered features functioned correctly without critical errors. According to the study's findings, the implementation of Scrum contributed to on-time delivery, improved development efficiency, and a final product that met user requirements and quality expectations. Specifically, the study reported a 100% completion rate of planned story points, indicating that Scrum's iterative cycles and frequent feedback loops helped the team manage scope and reduce rework.

The researchers concluded that the Scrum framework not only facilitated timely delivery but also enhanced product quality and team performance by maintaining close communication between the development team and stakeholders. By adopting Scrum, the project benefited from transparent progress tracking, adaptive planning, and incremental value delivery, all of which are critical to managing modern software projects where requirements often evolve over time. (Aurelio Nathanael Purnomo, 2025)

References

- Abdulrahman Alsari, A. M. (2023). Agile Recommendation System for Traditional Software Development Teams. *Journal of King Abdulaziz University: Computing and Information Technology Sciences*, 17 – 29.
- Adrielle Cristina Sassa, I. A. (2023). Scrum: A Systematic Literature Review. *International Journal of Advanced Computer Science and Applications*, 14.
- Asma Akhtar, B. B. (2022). EXTREME PROGRAMMING VS SCRUM: A COMPARISON OF AGILE MODELS. *International Journal of Technology, Innovation and Management*, 80–96.
- Atlassian. (6 November, 2025). *9 best agile project management tools for your team*. Retrieved from atlassian.com: <https://www.atlassian.com/agile/project-management/tools>
- Aurelio Nathanael Purnomo, R. S. (2025). Development of the “Smart Start” E-Commerce Website Using Scrum. *Journal of Business and Technology*, 1-10.
- Buehring, S. (7 May, 2024). *Lean software development implementation and practices*. Retrieved from agileKRC.com: <https://agilekrc.com/agile/lean/lean-software-development>
- Contributor, T. (26 April, 2021). *The origins of Agile software development*. Retrieved from TechTarget: <https://www.techtarget.com/whatis/reference/The-origins-of-Agile-software-development>
- Felix Temole, D. A. (2025). Agile Integration in Software Development: Principles, Practices, and Challenges. *SciencePG*, 18-29.
- Lynn, R. (27 October, 2025). *Guiding Principles of Lean Development*. Retrieved from Planview: <https://www.planview.com/resources/articles/lkdc-principles-lean-development>
- Maria Spichkova, K. I. (2025). Advanced approach for Agile/Scrum Process: RetroAI++. *Procedia Computer Science*, 1012-1021.
- Michal Hron, N. O. (2022). Why and how is Scrum being adapted in practice: A systematic review. *Journal of Systems and Software*, 183.
- Oleksandra Svintsytska, O. K. (2025). SPRINT PLANNING AND TASK ESTIMATION IN SCRUM PROJECTS USING JIRA TOOLS. *sj-economics scientific journal*, 28-44.
- Pham, T. (. (6 August, 2025). *What is Agile Methodology? Agile in Project Management Explained*. Retrieved from saigontechnology: <https://saigontechnology.com/blog/agile-methodology>

- Rania Aleryani, A. A. (2025). <https://doi.org/10.59628/jast.v3i6.1912> ISSN 2958-9568JASTVol.3, No.6, Page 1429- 1451, 2025Systematic Review of Agile DevelopmentMethodologies: Practices, Benefits,Implementation Challenges, and Future Directions. *Sana'a University Journal of Applied Sciences and Technology*, 1429- 1451.
- Rehkopf, M. (2025). *Kanban vs. scrum: Which agile are you?* Retrieved from Atlassian: <https://www.atlassian.com/agile/kanban/kanban-vs-scrum>
- Scrum.org. (6 February, 2026). *What is Scrum*. Retrieved from Scrum.org: <https://www.scrum.org/resources/what-scrum-module>
- Tersoo Hulugh, E. B. (2024). Agile Project Management in Developing Business-Centric Information Systems. *International Journal of Research Publication and Reviews*, 5083-5098.

Appendix

Some commands that I gave to AI to make the report more professional and organized.

1. “Make this Lean Software Development explanation clearer but keep it concise.”
2. “Rewrite this section to avoid repetition and improve clarity.”
3. “Help me structure the case study section for Scrum into clear subheadings.”
4. “Suggest how to organize the comparison between Scrum, XP, and Lean.”
5. “Create a summary table comparing the strengths of Scrum, XP, and Lean.”
6. “Format this section into bullet points suitable for academic writing.”
7. “Provide a brief explanation of Lean value stream mapping for diagram description.”

